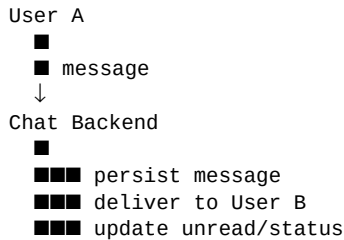


Chat System — Practical System Design Notes

Hinglish • WebSocket • Redis • Queue • DB • Scaling • Interview-focused

1. Chat System Kya Hai?

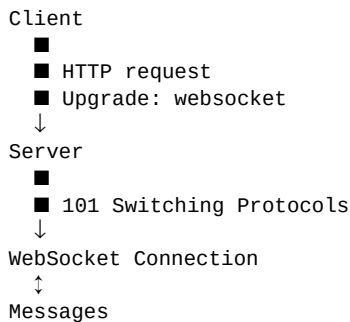
Chat system ka goal users ko messages reliably aur low-latency ke saath send/receive karwana hai. WhatsApp, Slack, Teams aur customer-support chat jaise systems isi problem family mein aate hain.



2. HTTP vs WebSocket

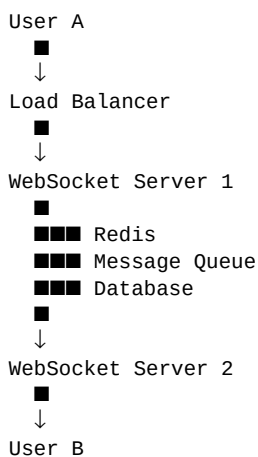
HTTP	WebSocket
Request/response model	Long-lived bidirectional connection
Client normally initiates request	Server can push data to client
Good for APIs	Good for real-time communication
Repeated polling may be needed	No continuous polling required for messages

3. WebSocket Connection



WebSocket connection establish hone ke baad same TCP connection par bidirectional messages exchange ho sakte hain.

4. Basic Chat Architecture



Single server se start karna simple hai, lekin multiple WebSocket servers par connected users ko coordinate karne ke liye shared infrastructure chahiye.

5. Single Server Problem

User A → Server 1
User B → Server 2

A sends message to B

Server 1 ko B ki connection
Server 2 par hai ye pata hai?

Need:
shared connection/presence information
and cross-server messaging

6. Redis Pub/Sub

User A
↓
WS Server 1
↓
Redis Pub/Sub
↓
WS Server 2
↓
User B

Redis Pub/Sub ek server se doosre server tak real-time event distribute karne mein useful hai. Lekin Pub/Sub ko durable message history ka replacement nahi samajhna chahiye.

7. Connection Registry

userId → websocketServerId

user-101 → ws-2
user-202 → ws-1
user-303 → ws-3

A sends to B:
lookup B
→ ws-1
→ route message to ws-1
→ deliver to B

Production design mein registry ka exact structure architecture ke scale aur infrastructure par depend karega. Redis shared state ke liye common choice hai.

8. Presence: Online / Offline

User connects
↓
presence = online
↓
heartbeat
↓
connection closes / timeout
↓
presence = offline

Presence ko permanently accurate assume nahi karna chahiye. Network failures aur mobile sleep ki wajah se heartbeat/TTL based state useful hoti hai.

9. Heartbeat / Ping-Pong

Client ↔ Server

PING
↓
PONG

No response for threshold
↓
connection considered dead
↓
cleanup presence

Heartbeat stale connections detect karne aur connection health maintain karne mein help karta hai.

10. Sending a Message

User A
↓
WebSocket Server
↓
authenticate/authorize
↓
validate message
↓
persist
↓
publish/deliver
↓
User B

Important decision: message ko DB mein persist karne ka point aur recipient ko delivery ka point carefully define karo.

11. Message Data Model

```
Message
{
  id,
  conversationId,
  senderId,
  receiverId / participants,
  body,
  type,
  createdAt,
  clientMessageId,
  sequence
}
```

Group chat ke liye receiverId ke bajay conversation participants model use ho sakta hai.

12. Conversation Model

```
Conversation
{
  id,
  type: "direct" | "group",
  createdAt
}
```

```
ConversationMember
{
  conversationId,
  userId,
  joinedAt,
  lastReadMessageId
}
```

13. Message Persistence

WebSocket delivery successful hone ke baad bhi message ko durable history mein rakhna important hota hai. Isse offline user later messages retrieve kar sakta hai aur reconnect ke baad history sync ho sakti hai.

14. Offline User

User B offline

A sends message

↓

DB persist

↓

B not connected

↓

message remains unread

B reconnects

↓

fetch messages after lastReadMessageId

↓

sync

15. Unread Count

Conversation

↓

lastReadMessageId

New messages after last read

↓

unread count

When user opens chat:

update lastReadMessageId

↓
unread = 0

16. Read Receipts

SENT
↓
DELIVERED
↓
READ

A sends message
B receives
B opens chat
B sends read event
A sees READ

Exact states product requirements par depend karte hain. Delivery aur read acknowledgement separate events ho sakte hain.

17. Duplicate Messages

Mobile/network retry ki wajah se client same logical message ko multiple times send kar sakta hai. Client-generated `clientId` se idempotency implement ki ja sakti hai.

```
clientId = abc-123
```

```
Request 1 → timeout
```

```
Request 2 → retry
```

```
Server:
```

```
already processed abc-123?
```

```
YES → return existing message
```

```
NO → create message
```

18. Message Ordering

Distributed systems mein timestamps alone ordering guarantee nahi karte. Per-conversation sequence number ya server-assigned ordering strategy useful ho sakti hai.

```
Conversation C1
```

```
seq 101
```

```
seq 102
```

```
seq 103
```

```
Client receives:
```

```
102
```

```
101
```

```
103
```

```
Can reorder using sequence.
```

19. At-Least-Once Delivery

Distributed delivery mein duplicate processing possible hoti hai. Isliye exactly-once ko blindly assume nahi karna chahiye; application-level idempotency important hai.

20. Message Queue

```
WebSocket Server
```

```
↓
```

```
Message Queue
```

```
↓
```

```
Workers / Consumers
```

```
↓
```

```
notifications, indexing,  
analytics, moderation etc.
```

Real-time delivery ke saath non-critical background work ko queue par offload kiya ja sakta hai.

21. WebSocket vs Queue

WebSocket	Queue
Real-time client communication	Asynchronous backend processing
Long-lived connection	Durable/buffered job/event flow depending on technology
User-facing delivery path	Workers and downstream services

22. Group Chat

Group G1

■■■ User A
■■■ User B
■■■ User C
■■■ User D

A sends

↓

fan-out / delivery

↓

B, C, D

Small groups mein straightforward fan-out practical ho sakta hai. Huge groups ke liye fan-out strategy, batching aur storage design carefully scale karna padta hai.

23. Fan-Out on Write vs Read

Fan-out on write	Message delivery/read model precomputed or pushed toward recipients
Fan-out on read	Recipients retrieve shared conversation data when reading

Trade-off: write amplification vs read complexity. Workload aur group size ke basis par choose karo.

24. Large Group / Celebrity Problem

Agar ek user ke message ko millions of recipients tak fan-out karna ho, naive synchronous fan-out expensive ho sakta hai. Queueing, batching, lazy fan-out aur specialized delivery strategies required ho sakti hain.

25. Reconnection

Connection lost

↓

client reconnects

↓

authenticate

↓

send last received sequence

↓

server returns missing messages

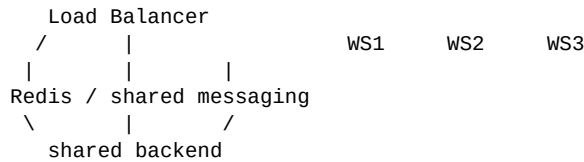
↓

resume real-time stream

26. Missed Message Sync

Client ko last successfully processed sequence/message ID maintain karna chahiye. Reconnect par server missing range return kar sakta hai.

27. Scaling WebSocket Servers



Horizontal scaling mein each server ke paas limited local connections hote hain. Cross-server message routing ke liye shared messaging/state layer required hoti hai.

28. Sticky Sessions

Sticky sessions same client ko same backend par route kar sakti hain, lekin ye cross-server coordination ka complete replacement nahi hain. Failure aur scaling behavior ko carefully consider karna chahiye.

29. Load Balancer and WebSocket

LB ko WebSocket upgrade/long-lived connections support karna chahiye. Idle timeout aur connection limits important configuration points hain.

30. Redis Role in Chat

Redis can support:

- presence
- connection/server mapping
- Pub/Sub or Streams depending on requirement
- rate limiting
- short-lived state
- caching

Durable chat history:

- DB / durable storage

31. Redis Pub/Sub Limitation

Pub/Sub mein subscriber disconnected ho to published message miss ho sakta hai. Durable processing/replay requirements ke liye Redis Streams, Kafka, RabbitMQ, SQS jaise alternatives depending on use case consider kiye ja sakte hain.

32. Database Choice

Chat message storage ke liye relational DB ya NoSQL dono possible hain. Choice message volume, query patterns, consistency requirements, partitioning strategy aur operational constraints par depend karti hai.

Common query: conversation ke latest messages efficiently fetch karna. Isliye conversationId + time/sequence based access pattern important hai.

33. Pagination

```
GET /conversations/C1/messages?before=seq_500&limit=50
```

Response:

```
seq 450 ... 499
```

Use cursor-based pagination
for large chat history.

Offset pagination large/high-churn chat tables mein inefficient ho sakti hai; cursor/sequence based pagination often better fit hoti hai.

34. Message Search

Large-scale full-text message search ke liye primary transactional DB ko directly search engine ki tarah use karna ideal nahi ho sakta. Async indexing pipeline se OpenSearch/Elasticsearch type system use kiya ja sakta hai, depending on requirements.

35. Attachments

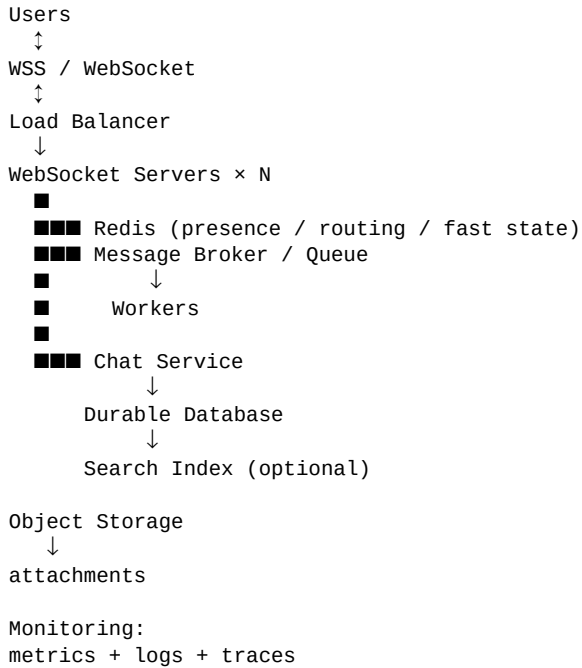
```
Client
↓
request upload authorization
↓
Object Storage
↓
message contains file metadata/URL
↓
recipient gets message
```

Large files ko WebSocket ke through stream karne ke bajay object storage + signed/presigned access pattern generally more scalable hota hai.

36. Security

- Authenticate WebSocket connection.
- Authorize conversation membership before sending/reading.
- Validate message size and type.
- Rate-limit abusive clients.
- Do not trust client-provided sender identity.
- Sanitize/render user content safely.
- Use TLS (wss) in production.
- Protect attachment access with authorization/signed URLs where appropriate.

37. Production Architecture



38. End-to-End Send Flow

1. User A sends message
2. WS server authenticates
3. Validate conversation membership
4. Generate/validate idempotency key
5. Persist message
6. Publish delivery event
7. Find recipient connection/server
8. Deliver to User B if online
9. Record delivery/read state as required
10. If offline → message remains for later sync
11. Background jobs handle notifications/analytics

39. Failure Scenarios

Failure	Handling
WebSocket server crashes	Reconnect client; recover missed messages from durable state
Redis unavailable	Depends on role; fail safely and recover shared state
User offline	Persist message; sync on reconnect
Duplicate send	clientMessageId/idempotency
Out-of-order messages	sequence/order metadata
Queue backlog	scale workers, inspect consumer lag/backlog
Slow DB	indexes, query tuning, caching/read strategy, scaling
Huge group	avoid naive synchronous fan-out; use scalable delivery strategy

40. Observability / Grafana

- Active WebSocket connections
- Connections per server
- Connection/disconnection rate
- Messages/sec
- Message delivery latency
- p50 / p95 / p99 latency
- Redis latency/errors
- Queue depth and consumer lag
- DB latency and slow queries
- Reconnect rate
- Delivery/read acknowledgement rate
- Error rate by operation

41. Interview Q&A;

Question	Short answer
WebSocket kyun?	Low-latency bidirectional real-time communication.
Multiple WS servers kaise?	Load balancer + shared routing/messaging state such as Redis.
Redis Pub/Sub kyun?	Cross-server real-time event propagation; not durable history.
Offline user?	Persist messages and sync missed messages after reconnect.
Duplicate message?	Client message ID/idempotency.
Ordering?	Per-conversation sequence/order metadata.
Read receipt?	Client sends read event; backend records state and broadcasts update.
Presence?	Heartbeat + TTL/state tracking.
Reconnect?	Re-authenticate and request messages after last acknowledged sequence.
Huge group?	Use scalable fan-out strategy; avoid naive synchronous fan-out.
Attachments?	Object storage; chat message stores metadata/reference.
Queue role?	Async processing such as notifications, indexing, analytics.
Sticky sessions?	Can help routing but don't eliminate distributed coordination.

42. Practice Scenarios

1. 100,000 users simultaneously connect. How will you scale WebSocket servers?
2. User A is on Server 1 and User B is on Server 2. A sends B a message. Explain the exact flow.
3. Redis Pub/Sub event is missed because a server disconnects. What design change is needed if replay is required?
4. User sends the same message twice because of a mobile network retry. Prevent duplicate messages.
5. Messages 103, 101, 102 arrive at the client. How do you preserve conversation ordering?
6. User goes offline for 2 hours and reconnects. How does message sync work?
7. Design read receipts for a one-to-one chat.
8. Design a group chat with 10,000 members.
9. Add image/file sharing without sending large files through WebSocket.
10. Grafana shows p99 message latency increasing. What would you inspect from client to DB?
11. Design unread counts for 1,000 conversations.
12. Design a Slack-like system with channels, direct messages and presence.

43. Local Node.js Practice

Practice project: Node.js + Express + ws/socket.io + Redis + BullMQ + MySQL/PostgreSQL/MongoDB. Start with one WebSocket server, then run 3 instances locally. Add Redis-based presence and cross-server message routing. Persist messages, add clientMessageId idempotency, sequence numbers, reconnect + missed-message sync, read receipts, and Prometheus/Grafana metrics.

Phase 1:
Client ↔ WS Server

Phase 2:
WS Server × 3
↓
Redis

Phase 3:
DB persistence

Phase 4:
Queue + notifications

Phase 5:
reconnect + ordering +
idempotency + metrics

44. Final Cheat Sheet

CHAT SYSTEM

```
Client
  ↑↓
WebSocket
  ↑↓
Load Balancer
  ↑↓
WS Servers × N
  ■
  ■■■ Redis → presence/routing/fast shared state
  ■■■ Queue → async processing
  ■■■ DB    → durable message history
```

Must handle:
→ authentication
→ authorization

- heartbeat
- reconnect
- offline sync
- ordering
- idempotency
- read receipts
- unread counts
- group fan-out
- rate limiting
- attachments
- monitoring

Golden interview line:

"I'll use WebSocket for real-time delivery, persist messages durably, use Redis for shared presence/routing state, support reconnect and sequence-based sync, add idempotency for duplicate sends, and use queues for non-critical asynchronous work."